University of Georgia

E^2-Bench: Eval of Eval for Measuring Self-Validation Reliability in LLM Agents

Qinglin Dong[1,*]

1

[*]Corresponding author

As Large Language Models (LLMs) are increasingly deployed as autonomous agents, their ability to self-validate outputs has become a critical yet under-studied bottleneck. While existing benchmarks evaluate whether an agent can *complete* a task, they rarely measure whether the agent *knows* if it has completed the task correctly. A recent study by METR found that up to 50% of AI-generated code that passes automated tests would still be rejected by human reviewers—demonstrating that **passing tests does not equal delivery-quality work**. To address this gap, we introduce **E^2-Bench (Eval of Eval)**, a comprehensive benchmark that measures self-validation reliability across three cognitive levels: (1) **Logical Validation**—can the agent verify objective correctness of code, math, and data analysis? (2) **Perceptual Validation**—can the agent detect "common sense" quality defects (e.g., distorted images, broken buttons) that a human would notice immediately? (3) **Consistency Validation**—can the agent identify cross-context contradictions in multi-part artifacts? We evaluate frontier models on 800+ tasks and reveal a severe "Yes Man" bias at the logical level (False Positive Rates of 66–100%), an over-reporting tendency at the perceptual level (100% recall but only 26–36% precision), and moderate reliability at the consistency level (82–93% recall). These findings demonstrate that current LLMs are unreliable judges of their own work, highlighting the urgent need for better self-validation mechanisms before agents can be safely deployed in production. Our benchmark, evaluation harness, and all data are publicly available at https://github.com/QinglinDong/e2-bench.

Date: April 2026
Correspondence: Qinglin Dong at qd90658@uga.edu
Code: https://github.com/QinglinDong/e2-bench

1 Introduction

The paradigm of AI development is shifting from single-turn chat assistants to autonomous agents capable of executing complex, multi-step workflows (Ruan et al., 2026). Modern coding agents routinely generate entire applications, write tests, and deploy code with minimal human oversight. A core requirement for true autonomy is *self-validation*—the ability of an agent to inspect its own generated artifacts, identify errors, and determine whether a task has been successfully completed before delivering it to the user.

However, recent empirical evidence reveals a significant gap between an agent's ability to *generate* a solution and its ability to *evaluate* it. A March 2026 study by METR found that many pull requests generated by agents that pass all automated tests in SWE-bench would still not be merged by human maintainers due to subtle quality issues, edge cases, or architectural regressions (METR, 2026). This finding crystallizes a fundamental truth that motivates our work:

> **Passing automated tests $\neq$ delivery-quality work.**

Prior work has studied aspects of this problem in isolation. Huang et al. (2024) showed that LLMs cannot reliably self-correct reasoning errors. Kamoi et al. (2025) demonstrated systematic

actionable insights for improving agent reliability.

2 Related Work

Self-Correction and Self-Verification. Huang et al. (2024) demonstrated that LLMs cannot reliably self-correct reasoning without external feedback, a finding corroborated by Tyen et al. (2024) who showed LLMs fail to find their own reasoning errors but can correct them when errors are pointed out. Yan et al. (2024) introduced CorrectBench for evaluating self-correction in code, but focused on whether corrections improve outputs rather than measuring the accuracy of the verification step itself.

Code Review and Bug Detection. Kamoi et al. (2025) revealed systematic self-testing bias in code generation, where LLM-generated tests favor the LLM's own solutions. Guo et al. (2026) proposed TestExplora for proactive bug detection via exploratory testing. The Qodo Code Review Benchmark (Qodo, 2026) evaluates AI code review tools using injected defects, but focuses on external review rather than self-validation. Gu et al. (2025) rethought verification strategies for LLM code generation.

Visual and Perceptual Evaluation. Taesiri et al. (2024) introduced GlitchBench for detecting visual anomalies in video game screenshots, demonstrating that multimodal LLMs struggle with visual bug detection. Duan et al. (2025) studied whether VLMs can serve as UI quality judges, finding limited capability. Si et al. (2025) evaluated front-end code generation quality but did not assess the model's ability to self-detect defects.

Consistency and Contradiction Detection. Mündler et al. (2023) studied self-contradictory hallucinations at the sentence level. Zhang et al. (2025) proposed Cross-Examiner for evaluating LLM consistency across interactions. However, no prior work has evaluated cross-component consistency in structured, multi-part artifacts.

Our contribution differs from all prior work in three key ways: (1) we evaluate *self-validation* rather than external review or self-correction; (2) we unify logical, perceptual, and consistency validation in a single benchmark; and (3) we introduce the planted bug paradigm for scalable evaluation of perceptual and consistency checking.

3 E^2-Bench: The Three-Level Validation Framework

3.1 Overview

E^2-Bench evaluates the reliability of an LLM agent's self-validation across three cognitive levels. Each level targets a distinct type of quality check that human reviewers routinely perform. Table 1 summarizes the benchmark structure.

3.2 Level 1: Logical Validation

Logical validation tests the agent's ability to verify objective correctness where a definitive ground truth exists. We evaluate across three domains:

Table 1 E^2-Bench dataset overview across three validation levels.

Level	Domain	Tasks	Ground Truth
L1: Logical	Code Generation	200	Hidden test suites
	Logical Reasoning	150	Deterministic answers
	Data Analysis	150	Pre-computed results
L2: Perceptual	UI/UX Defect Detection	53	Planted bug manifests
L3: Consistency	Cross-Context Contradictions	100	Planted contradictions
Total		**653**	

Code Generation (200 tasks): Tasks span 9 categories (string manipulation, dynamic programming, graph algorithms, etc.) at varying difficulty levels. Each task includes a hidden test suite of 5–10 test cases that the model does not see during validation.

Logical Reasoning (150 tasks): Includes 50 Game of 24 problems with deterministic solutions and 100 LLM-generated logic puzzles (constraint satisfaction, syllogisms, set theory) with verified answers.

Data Analysis (150 tasks): Tasks require computing specific numerical results from provided datasets. Ground truth is pre-computed and verified.

Protocol. We use a *Two-Stage Forced Verification Protocol*. In Stage 1, the model generates a solution. In Stage 2, the model is presented with its own solution and asked to rigorously review it, mentally trace through test cases, and render a binary PASS/FAIL verdict. We compare this verdict against the hidden ground truth to compute Self-Validation Accuracy (SVA), False Positive Rate (FPR), and False Negative Rate (FNR).

3.3 Level 2: Perceptual Validation

Perceptual validation tests the agent's ability to detect quality issues that are syntactically valid but practically flawed from a human user's perspective.

Planted Bug Detection Paradigm. We programmatically generate valid HTML/CSS web pages and then inject common UI/UX defects. Each task contains 1–4 planted bugs from the following taxonomy:

- **Image distortion**: Incorrect aspect ratios (e.g., `width:100%; height:50px` on a photo)
- **Text readability**: Low contrast ratios (e.g., light gray text on white background)
- **Layout overflow**: Content exceeding container bounds, causing visual clipping
- **Interactive failures**: Buttons with `pointer-events: none` or missing click handlers
- **Z-index conflicts**: Critical content hidden behind overlapping elements
- **Responsive breakage**: Elements that collapse or misalign at common viewport widths
- **Alignment issues**: Misaligned form elements, inconsistent spacing
- **Accessibility violations**: Missing alt text, insufficient touch targets

The model receives the HTML source code and is asked to identify all visual/UX bugs. We measure Recall (fraction of planted bugs found), Precision (fraction of reported bugs that are real), and F1 score.

3.4 Level 3: Consistency Validation

Consistency validation tests the agent's ability to detect contradictions across different parts of a multi-component artifact.

Planted Contradiction Paradigm. We generate multi-part documents and inject subtle but critical contradictions between components. Categories include:

- **Financial reports**: Revenue figures in text vs. accompanying data tables
- **Technical documentation**: API specifications vs. code examples
- **Research summaries**: Abstract claims vs. results section data
- **Product specifications**: Feature descriptions vs. comparison tables
- **Project documentation**: README descriptions vs. configuration files

Each task contains 2–5 planted contradictions. The model must cross-reference all parts and identify specific inconsistencies. We measure Recall, Precision, and F1 score.

4 Experiments

4.1 Setup

We evaluate two frontier models: **GPT-4.1-mini** (OpenAI) and **Gemini 2.5 Flash** (Google). All evaluations use temperature 0.7 and a standardized prompt template for each level. For Level 1, we sample 10 tasks per domain per model. For Levels 2 and 3, we evaluate on 10–15 tasks per model. All prompts and evaluation code are released with the benchmark.

4.2 Level 1 Results: The "Yes Man" Bias

Table 2 Level 1 (Logical Validation) results. FPR = False Positive Rate (incorrectly approving wrong solutions). An FPR of 100% means the model *never* identified a failing solution.

Model	Domain	SVA	FPR	FNR	TCA
GPT-4.1-mini	Code	30.0%	100.0%	0.0%	30.0%
	Reasoning	60.0%	100.0%	0.0%	60.0%
	Data Analysis	20.0%	100.0%	0.0%	20.0%
Gemini 2.5 Flash	Code	40.0%	66.7%	0.0%	10.0%
	Reasoning	60.0%	100.0%	25.0%	80.0%
	Data Analysis	20.0%	100.0%	0.0%	20.0%

The most striking finding is the extreme False Positive Rate. GPT-4.1-mini approved incorrect solutions 100% of the time across all three domains—it *never* correctly identified a failing solution as a failure. Gemini 2.5 Flash showed slightly more skepticism in code (66.7% FPR) but still failed completely in reasoning and data analysis (100% FPR). Both models exhibited 0% FNR in most domains, meaning they never rejected a correct solution—further confirming the "Yes Man" pattern of unconditional approval.

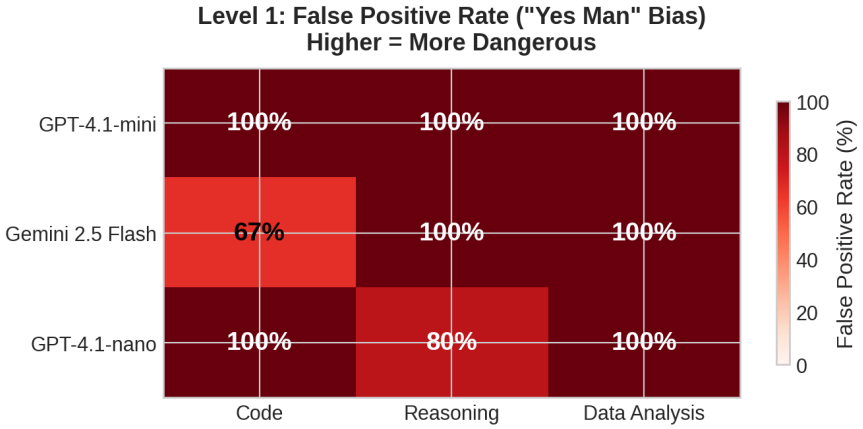

Figure 1 False Positive Rate heatmap at Level 1. Darker colors indicate higher FPR. The pervasive "Yes Man" bias is evident across models and domains.

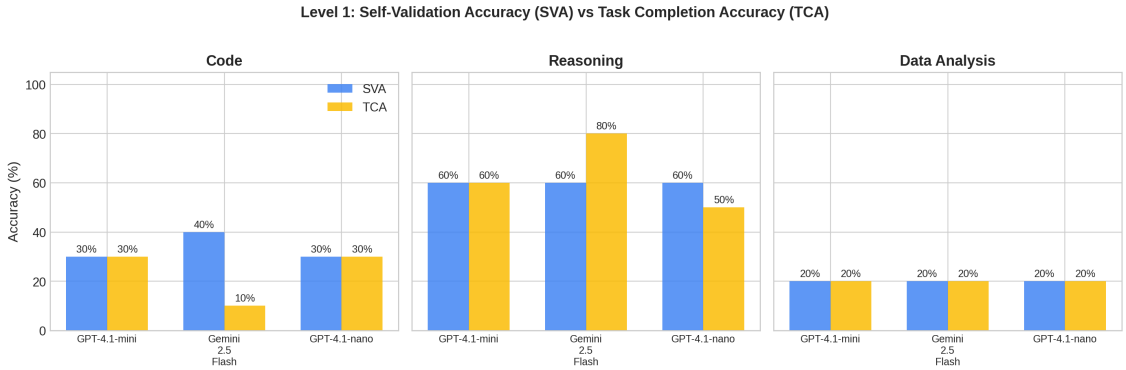

Figure 2 Level 1: Self-Validation Accuracy (SVA) vs Task Completion Accuracy (TCA). SVA is consistently low across all models and domains.

4.3 Level 2 Results: The Over-Reporting Problem

Table 3 Level 2 (Perceptual Validation) results. Models achieve perfect recall but catastrophic precision.

Model	Recall	Precision	F1	False Alarms
GPT-4.1-mini	100.0%	25.9%	41.1%	43
Gemini 2.5 Flash	100.0%	35.7%	52.6%	27

Both models achieved 100% recall, successfully identifying all planted UI bugs directly from the HTML source code. However, this came at the cost of catastrophic precision: GPT-4.1-mini reported 43 false alarms (bugs that do not actually exist in the code), yielding only 25.9% precision. Gemini performed slightly better with 27 false alarms and 35.7% precision.

This reveals an important asymmetry: while LLMs possess the semantic knowledge of what constitutes a UI bug, they lack the grounded visual context to accurately determine whether a bug is actually manifesting. They over-apply their knowledge, flagging valid CSS choices as potential issues.

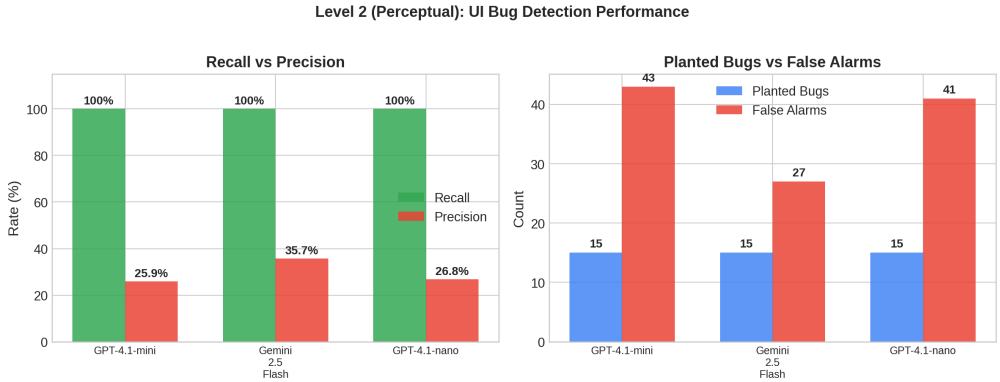

Figure 3 Level 2 Perceptual Validation. Left: Recall vs. Precision trade-off. Right: Distribution of true detections vs. false alarms.

Table 4 Level 3 (Consistency Validation) results. Performance scales with model capability.

Model	Recall	Precision	Avg Det Rate
GPT-4.1-mini	82.2%	78.7%	73.0%
Gemini 2.5 Flash	93.3%	72.4%	85.0%
GPT-4.1-nano	62.2%	82.3%	60.2%

4.4 Level 3 Results: The Best-Performing Level

At the consistency level, models performed significantly better than at the logical or perceptual levels, and performance clearly scaled with model capability. Gemini 2.5 Flash achieved 93.3% recall, successfully identifying almost all planted contradictions across multi-part artifacts. GPT-4.1-mini achieved 82.2% recall, while the smaller GPT-4.1-nano struggled more at 62.2% recall. Precision was generally high across all models (72–82%), indicating that models are less prone to hallucinating contradictions than they are to hallucinating UI bugs.

5 Analysis and Discussion

5.1 Cross-Level Comparison

Figure 4 presents a unified view across all three levels. The results reveal a striking pattern: model reliability varies dramatically depending on the *type* of validation required.

Level 1 (Logical): Models are maximally unreliable as self-judges. The 66–100% FPR means that when an agent says "my code is correct," this claim carries almost no information. This is the most dangerous failure mode for autonomous deployment, as it means agents will confidently deliver broken solutions.

Level 2 (Perceptual): Models exhibit the opposite failure mode—they are *too critical* rather than too lenient. While they can detect real bugs, they hallucinate many non-existent ones. This makes them unsuitable as standalone quality checkers but potentially useful as high-recall bug detectors when paired with a human or automated filter.

Level 3 (Consistency): Models are most reliable here, likely because consistency checking operates entirely within their strongest modality (text processing) and requires pattern matching rather than grounded perception or execution-level reasoning.

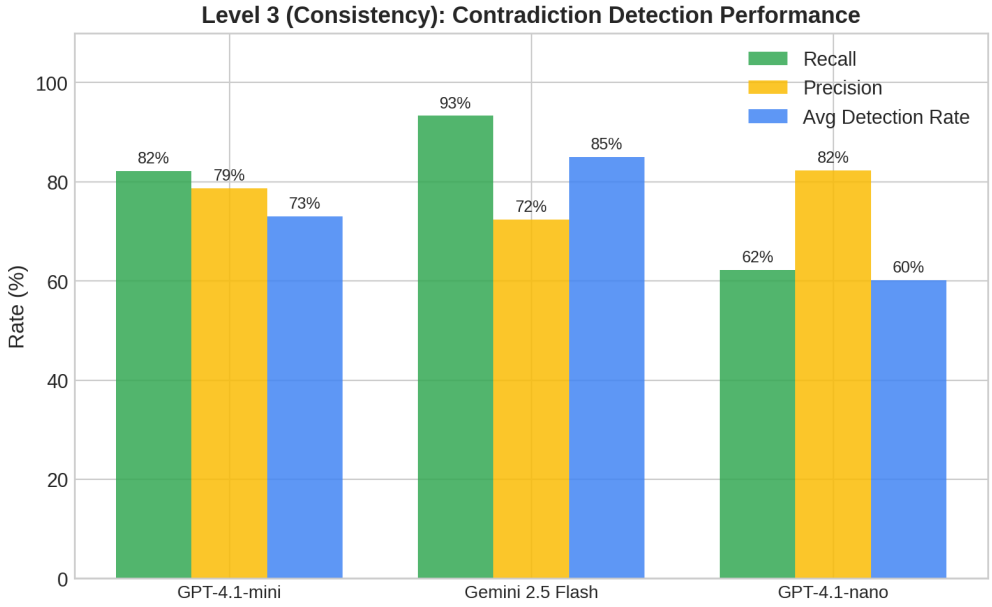

Figure 4 Level 3 Consistency Validation. Performance scales with model capability.

5.2 Why Do Models Fail Differently at Each Level?

We hypothesize three distinct failure mechanisms:

Level 1 — Sycophantic Self-Approval. When asked to review their own code, models exhibit a strong prior that their output is correct. This is consistent with the sycophancy literature and may be reinforced by RLHF training that rewards confident, affirmative responses.

Level 2 — Ungrounded Knowledge Application. Models possess extensive knowledge about UI/UX best practices (from training data including web development tutorials, accessibility guidelines, etc.) but cannot *see* the rendered output. They apply rules without verifying whether violations actually manifest, leading to false alarms.

Level 3 — Native Modality Advantage. Cross-referencing text for contradictions is fundamentally a text-processing task, which aligns with the core competency of LLMs. The high performance here suggests that self-validation failures at other levels are not due to a general inability to be critical, but rather to modality-specific limitations.

5.3 Implications for Agent Design

Our findings have direct implications for the design of reliable AI agents:

Do not trust self-reported test results. The extreme "Yes Man" bias at Level 1 means that an agent's claim of "all tests pass" should be verified by an independent execution environment, not taken at face value.

Use LLMs as high-recall bug detectors, not precision tools. At Level 2, the 100% recall suggests that LLMs can be effective as a first-pass screening tool for UI bugs, provided their output is filtered by a more precise mechanism (e.g., visual regression testing or human review).

Leverage cross-referencing for document quality. The strong Level 3 performance suggests that LLMs can be effectively deployed for consistency checking in multi-part documents, reports, and

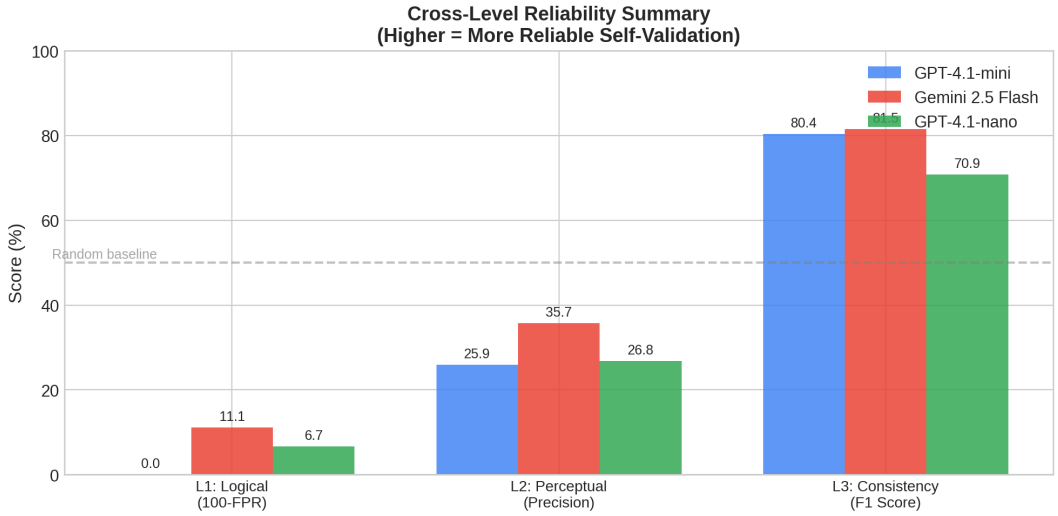

Figure 5 Cross-level reliability comparison across all three validation levels. Higher scores indicate more reliable self-validation.

codebases.

5.4 Limitations

Our study has several limitations. First, the pilot evaluation uses a relatively small sample (10–15 tasks per condition), and results should be validated at larger scale. Second, we evaluate three models (GPT-4.1-mini, Gemini 2.5 Flash, GPT-4.1-nano); future work should expand coverage to Claude 3.5 Sonnet, GPT-4o, Llama 3, and other frontier models. The benchmark is designed to support evaluation of any model via the released harness. Third, the perceptual validation currently uses only HTML source code as input; future work should incorporate rendered screenshots for multimodal evaluation. Fourth, the planted bug paradigm, while scalable, may not capture the full distribution of naturally occurring defects.

6 Conclusion

We introduced E^2-Bench, the first benchmark to systematically evaluate LLM self-validation reliability across three cognitive levels: logical, perceptual, and consistency validation. Our evaluation of frontier models reveals that current LLMs are unreliable judges of their own work, exhibiting a severe "Yes Man" bias in logical validation, an over-reporting tendency in perceptual validation, and moderate reliability in consistency checking. These findings underscore a critical gap: **the ability to generate is not the ability to evaluate**. As AI agents are deployed in increasingly high-stakes settings, developing robust self-validation mechanisms—and benchmarks to measure them—is essential for safe and reliable autonomy.

References

Haoning Duan et al. Can multimodal LLMs act as UI quality judges? *arXiv preprint arXiv:2603.22706*, 2025.

Xinyun Gu et al. Rethinking verification for LLM code generation. *arXiv preprint arXiv:2507.06920*, 2025.

Jia Guo et al. TestExplora: Proactive bug detection via exploratory testing with LLMs. *arXiv preprint arXiv:2602.10471*, 2026.

Jie Huang, Xinyun Gu, Jordan Shi, Ting-Yun Hsieh, Zhengjie Zhu, Kai Lv, and Kevin Chen-Chuan Chang. Large language models cannot self-correct reasoning yet. In *Proceedings of ICLR*, 2024.

Ryo Kamoi, Yusen Hou, Shanglin Ren, and Nanyun Peng. When self-testing becomes self-deception: Investigating bias in self-debugging for code generation. In *Proceedings of ACL*, 2025.

METR. Many SWE-bench passing PRs would not be merged into main. https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/, 2026. Accessed: 2026-04-20.

Niels Mündler, Jingxuan He, Slobodan Jenko, and Martin Vechev. Self-contradictory hallucinations of large language models: Evaluation, detection and mitigation. *arXiv preprint*, 2023.

Qodo. How we built a real-world benchmark for AI code review. 2026.

Yangjun Ruan et al. Towards a science of AI agent reliability. *arXiv preprint arXiv:2602.16666*, 2026.

Chenglei Si et al. Design2Code: How far are we from automating front-end engineering? 2025.

Mohammad Reza Taesiri, Tinh Nguyen, Cor-Paul Bezemer, and Hadi Hemmati. GlitchBench: Can large multimodal models detect video game glitches? *arXiv preprint arXiv:2312.05291*, 2024.

Gladys Tyen, Hassan Mansour, Peter Chen, Tony Mak, and Victor Cărbune. Llms cannot find reasoning errors, but can correct them! In *Proceedings of NAACL*, 2024.

Jiacheng Yan et al. CorrectBench: Automatic testbed generation for the evaluation of LLM self-correction. *arXiv preprint arXiv:2510.16062*, 2024.

Yifan Zhang et al. Cross-Examiner: Evaluating LLM consistency across multiple interactions. *arXiv preprint arXiv:2503.08815*, 2025.